

COMP2054 ADE

Lec 15

Priority Queues & Heaps

Reminder: ADT and CDT

Terminology:

- Abstract Data Type (ADT)
 - Only concerned with specifying the interface
 - “Behaviour as seen from the outside”
 - Specifies the methods provided - and possibly requirements on their complexity, e.g. “must run in $O(\log n)$ ”
- Concrete Data Type (CDT)
 - Some structure and algorithm used for an actual implementation
 - “Behaviour as seen from the inside”

Reminder: ADT and CDT utility

Standard object-oriented reasons:

- Separation of the “specification” from the “implementation details”
- Allows different CDTs to be explored for the same ADT
- Quickly swap or improve CDTs without users of the ADT having to change their code
- **Watch out for: difference between**
 - **names used for ADTs**
 - **names used for associated CDTs**

ADT / CDT

In this lecture:

- ADT= “Priority Queue” (PQ)
- CDT= “Binary Heap”
 - or “Minheap” to be more precise

“Priority Queue”: Idea and usage

- Usage: collecting together entries and treating them as a queue but with each element having a priority
 - Higher priority items are retrieved first
- Used in many queuing systems
 - E.g. in operating systems for queuing of jobs
- PQs are internally in many algorithms
 - e.g. within Dijkstra’s algorithm, and also A^*

“Priority Queue” ADT

- The PQ stores multiple elements, each in the form of a “Key-Value pair” (K,V)
 - Key K
 - this is (typically) a number representing the priority, and we take smaller numbers to be higher priority
 - Generally, the key can be anything that can have a total order, but this does not affect the workings of a heap – so we use integers for simplicity
 - Value V
 - This can be any associated value – e.g. could be the ID of a person
 - The value itself plays no role in the functioning of the heap, and so will typically ignore it
- Note that a heap is generally allowed to have multiple entries with the same key

PQ – main operations

Main operations are

- `void add(K k, V v)`
 - add the Key-Value pair (k,v) to the storage
- `(K, V) removeMin()`
 - Removes and returns a K,V pair in which the key is guaranteed to be a minimum within all the keys

PQ – other operations

Also typically have

- `int size()`
 - Returns the number of Key-Value pairs stored
- `(K,V) peekMin()`
 - returns a K,V pair in which the key is guaranteed to be a minimum within all the keys – but leaves the entry in place
- And others

PQ: using lists

- As in many cases, we can do a first implementation using lists
- If the list is unsorted:
 - $\text{Add}(k)$ is $O(1)$ – we can just put the entry at the start
 - $\text{removeMin}()$ is $O(n)$ as we must scan the list for the minimum
- If the list is sorted (smaller first):
 - $\text{add}(k)$ is $O(n)$ as must scan to find the correct position to insert it
 - removeMin is $O(1)$ as know the min is at the head

General Remark

- “Some operations are $O(1)$ and others $O(n)$, so the average is not too bad” ??
 - NO NO NO!!!
- The average could still be $O(n)$
- Suppose, that
 - 99% take time 1
 - 1% take time n

The average $(n + 99)/100$ is still $O(n)$
- This only improves if the worst-case operations are ‘rare’, otherwise the expensive operations still dominate
 - E.g. see the study of amortised complexity. If the resizing is “rare” then it does not dominate the cost.

Aim:

- Can we find an implementation of a PQ such that **all** operations are better than $O(n)$?
 - I.e. want all operations $o(n)$.
 - E.g. aim for them all to be $O(\log n)$
- We can do this by implementing a Priority Queue using a kind of binary tree called a 'heap':

PQ: improve upon $O(n)$

- How would we organise the tree?
- We want a minimum element to be easy to access
 - It hence makes reasonable sense to place it at the root
 - Then we want the same kind of idea for all the subtrees
 - In all subtrees the root should be a minimum
 - This idea leads to heaps:

PQ using heaps

- There are different kinds of heaps, but the common property is a numeric one that leads to root being a smallest key
- Crucial numeric property of heaps – can be said in multiple ways:
 - For every node: the key of the node must not be smaller than the key of the parent
 - That is, for every node the children (if there are any) cannot have a smaller key
 - For all paths from the root to any leaf, the sequence is non-decreasing

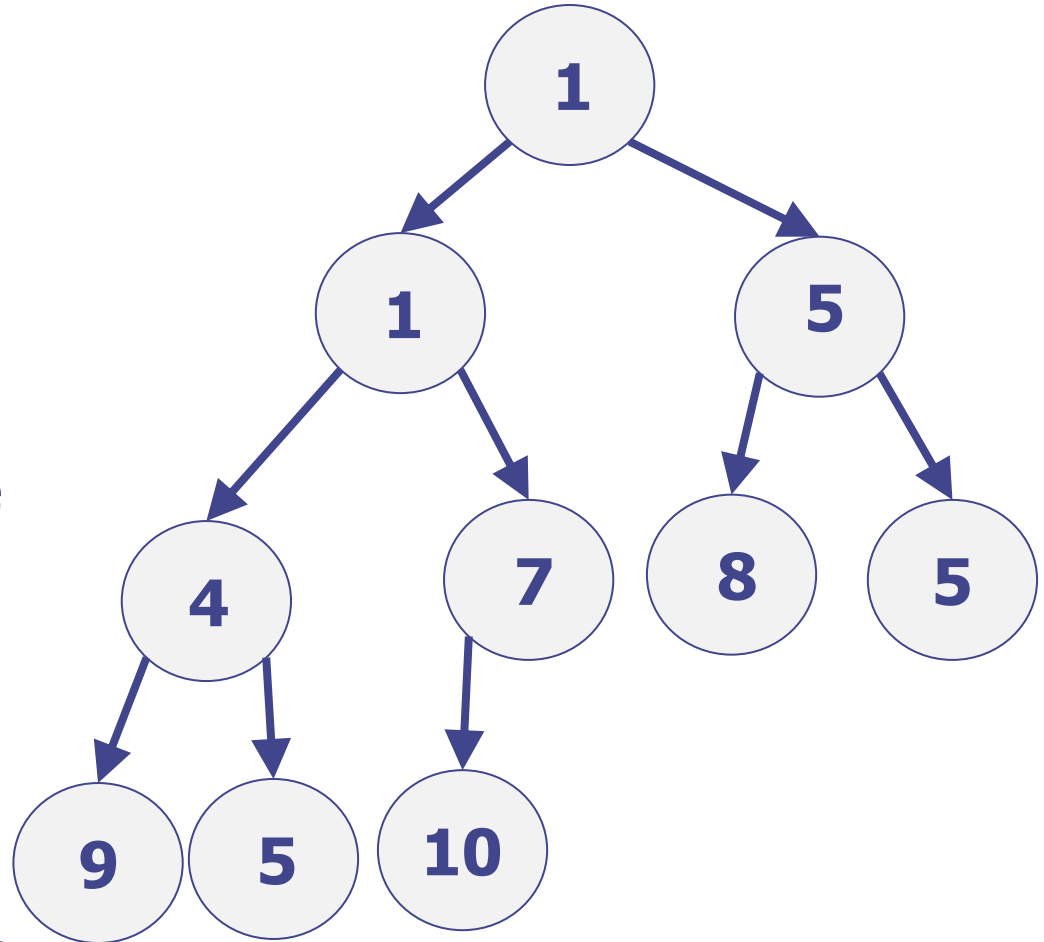
Standard Heaps

- Heaps also have some restrictions on the “shape of the binary tree”
- The standard restriction (as used in this module) is that
- The tree must be “complete”
 - Every level must be full, except possibly the last level, and in this case all the nodes must be as far left as possible

Heap Example

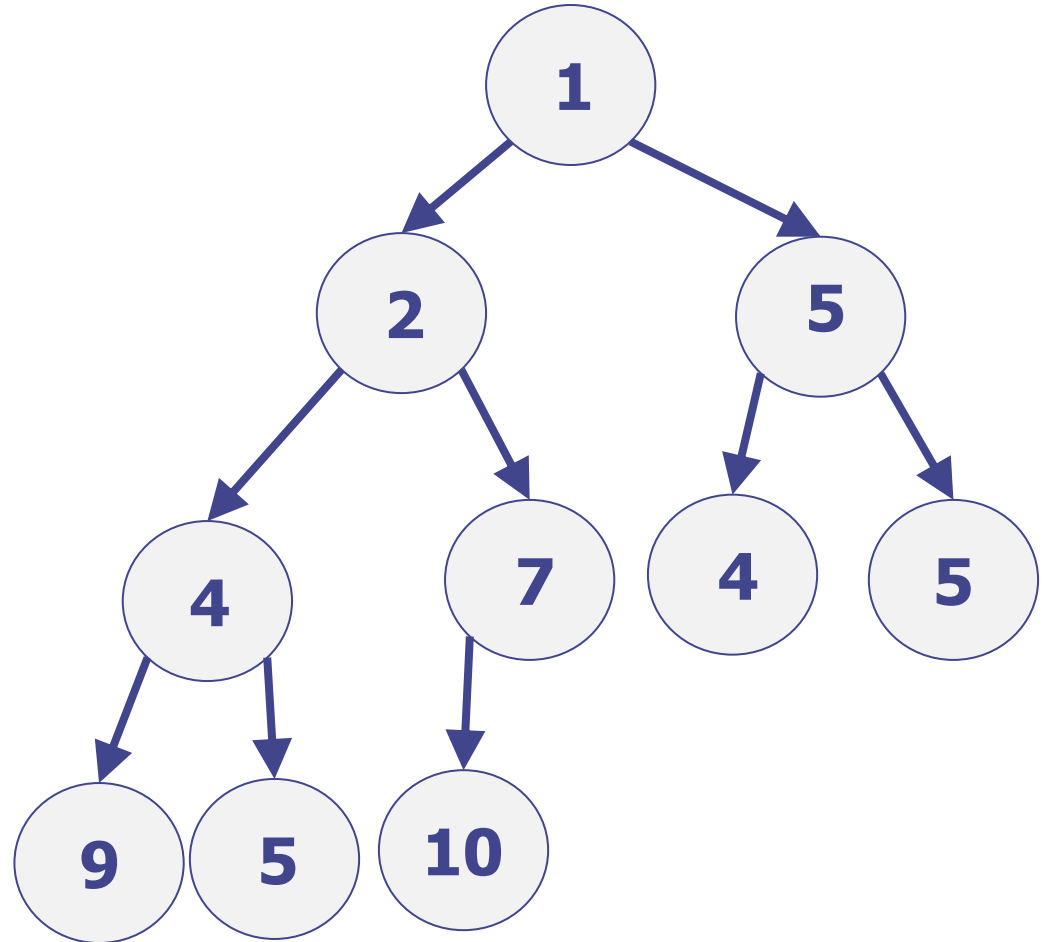
- This tree is a heap as children are never smaller than their parent and the tree is complete
- Recall: A tree is "**complete**" if every level, except the last, is as full as possible. The last level is allowed to be incomplete, but every node must be as far left as possible

The rightmost node of the last level is called the "**last node**". In this case it stores the key=10



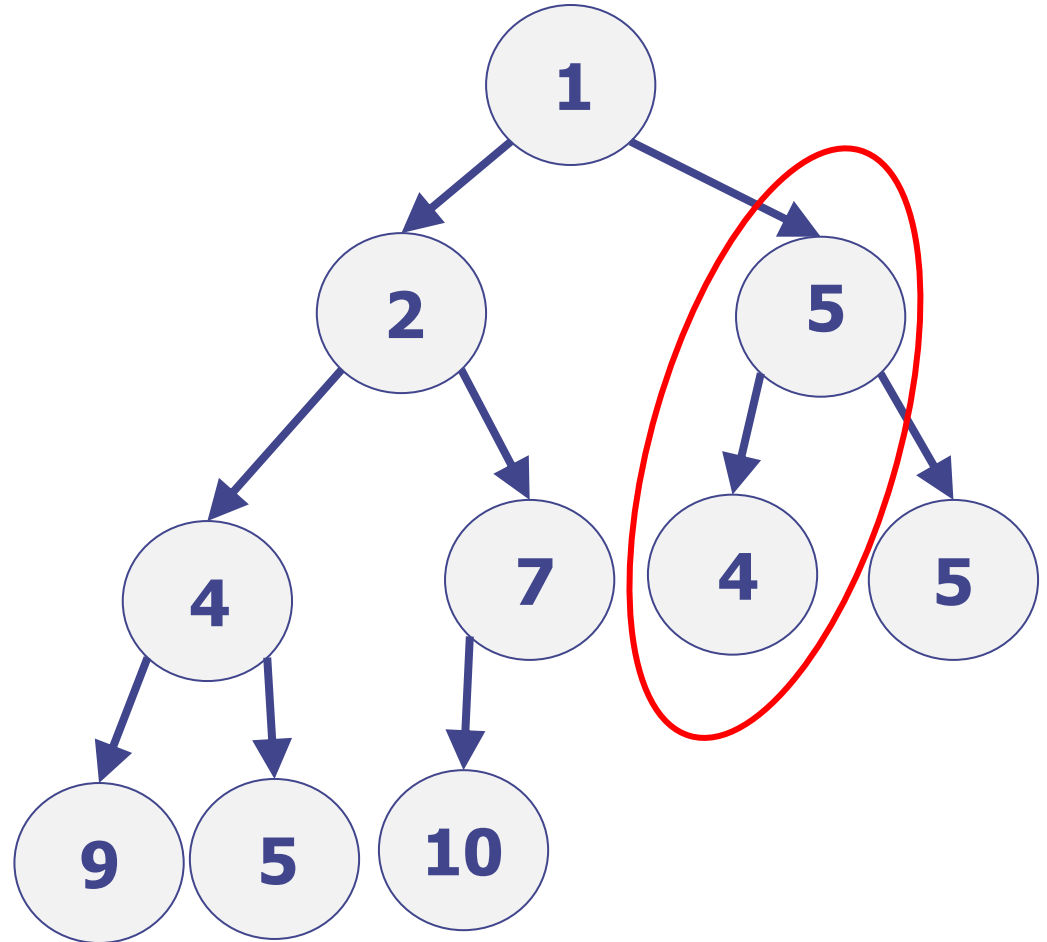
Is this a Heap ?

- Why?



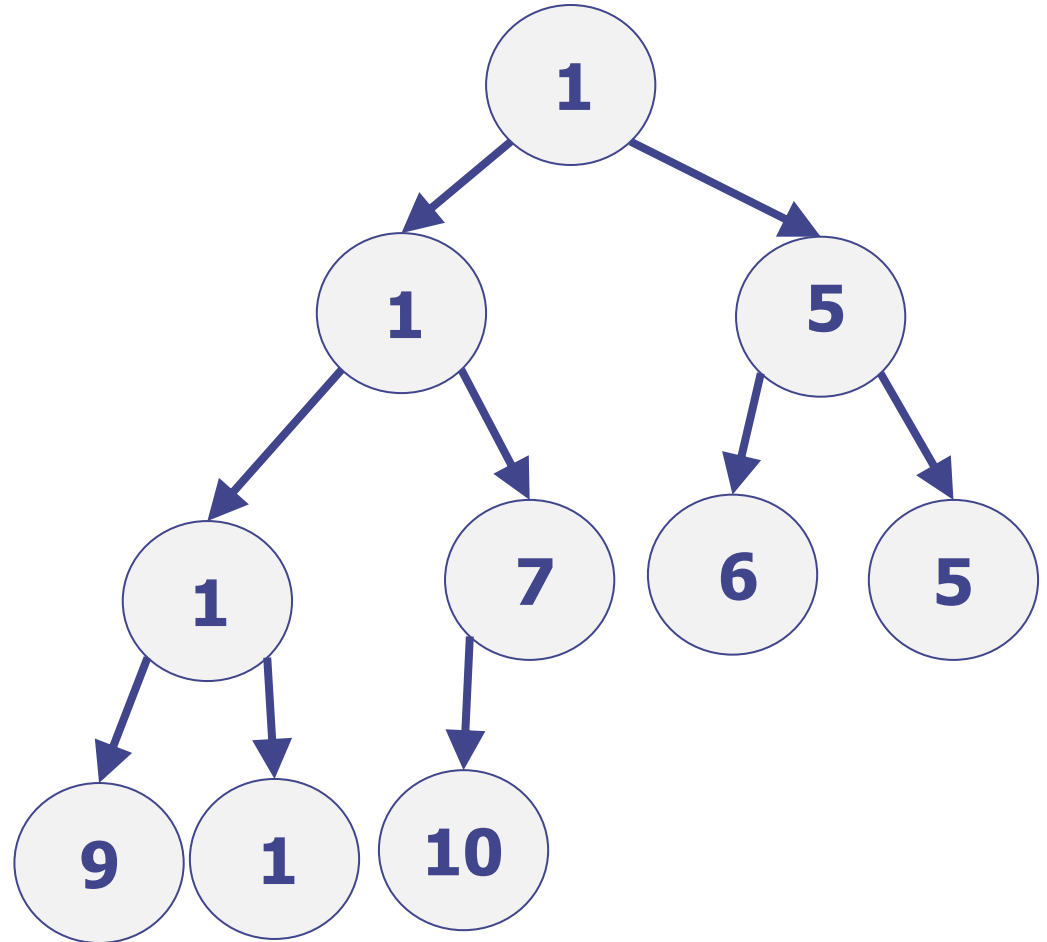
Is this a Heap ?

- No
- Bad ordering



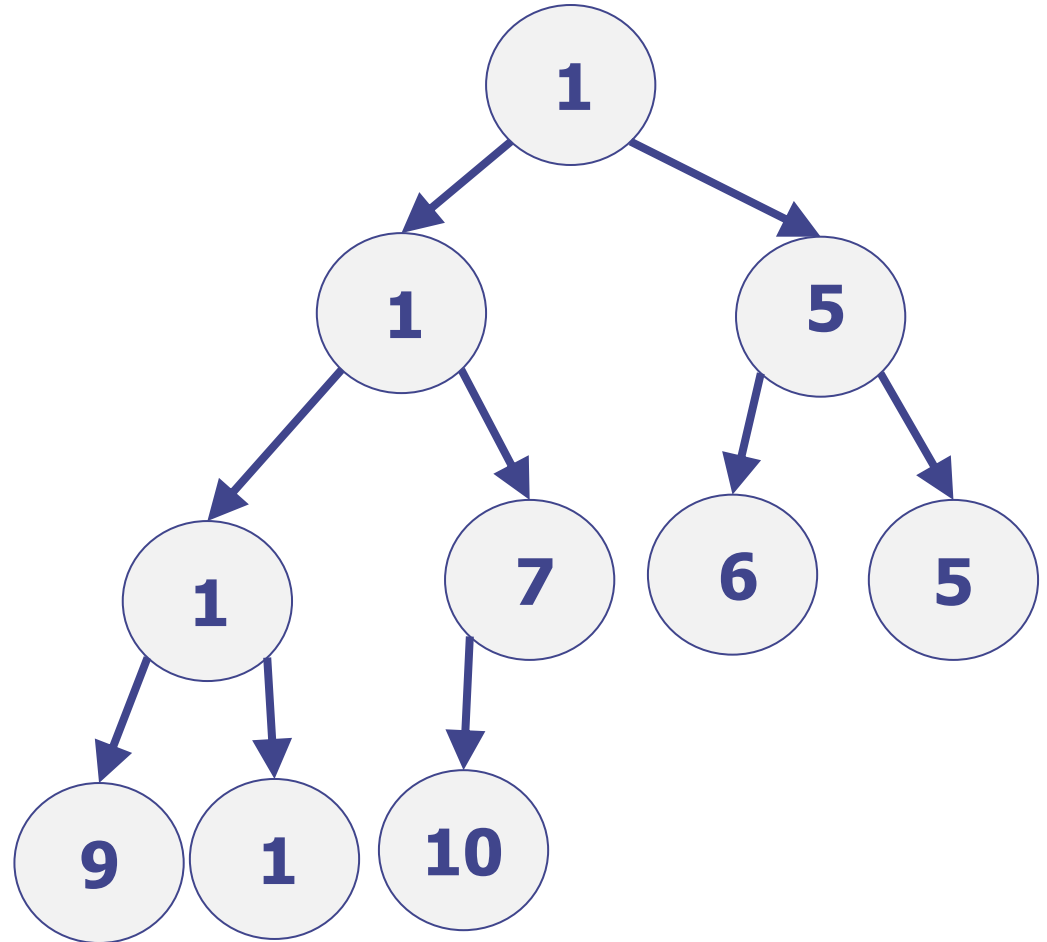
Is this a Heap ?

- Why?



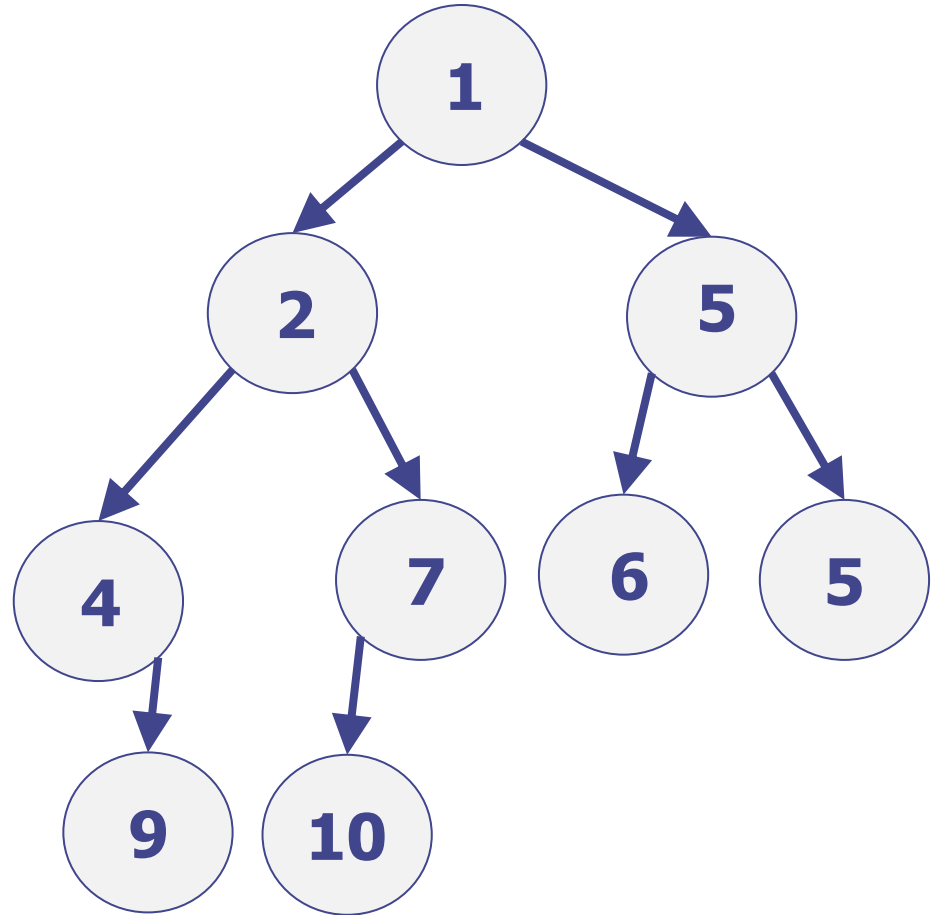
Is this a Heap ?

- Yes?
- It is allowed to have multiple nodes with the same value as the root
- Hence always say
- “removeMin” returns **a** minimum, not **the** minimum



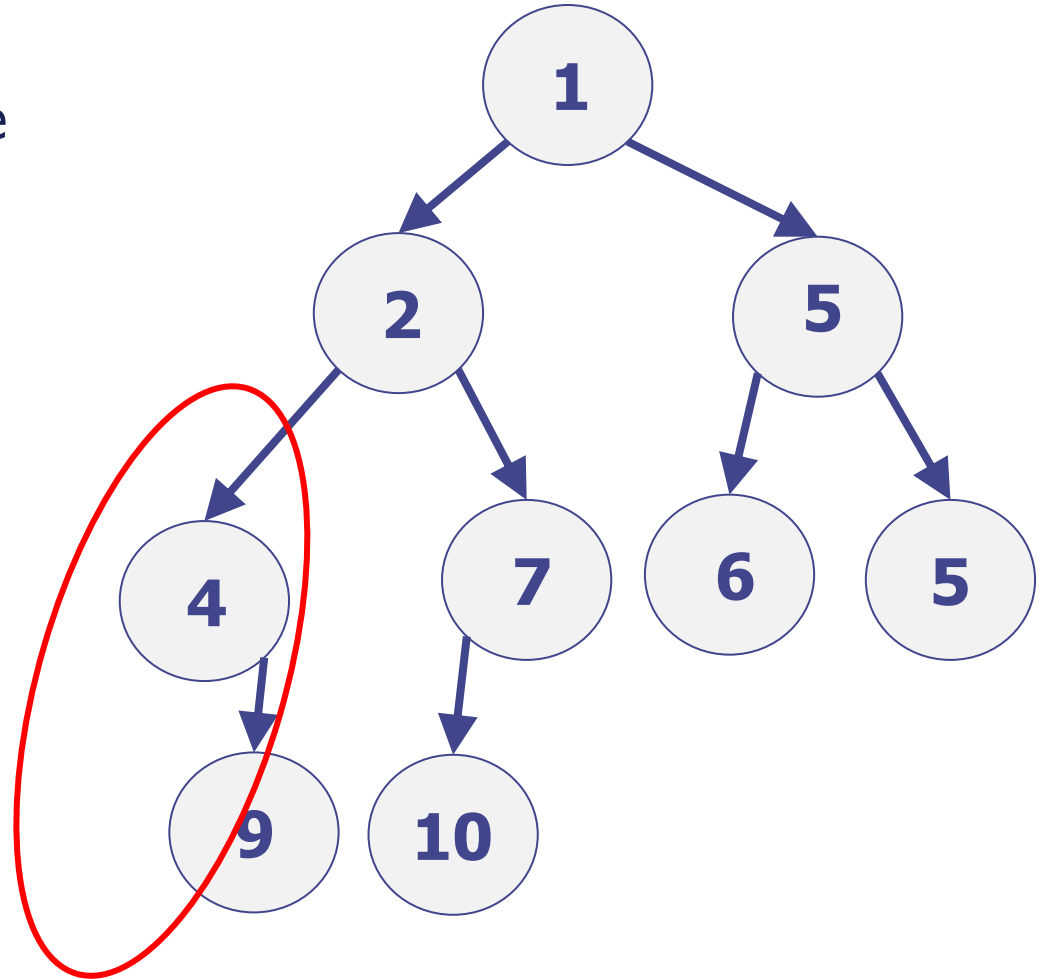
Is this a Heap ?

- Why?



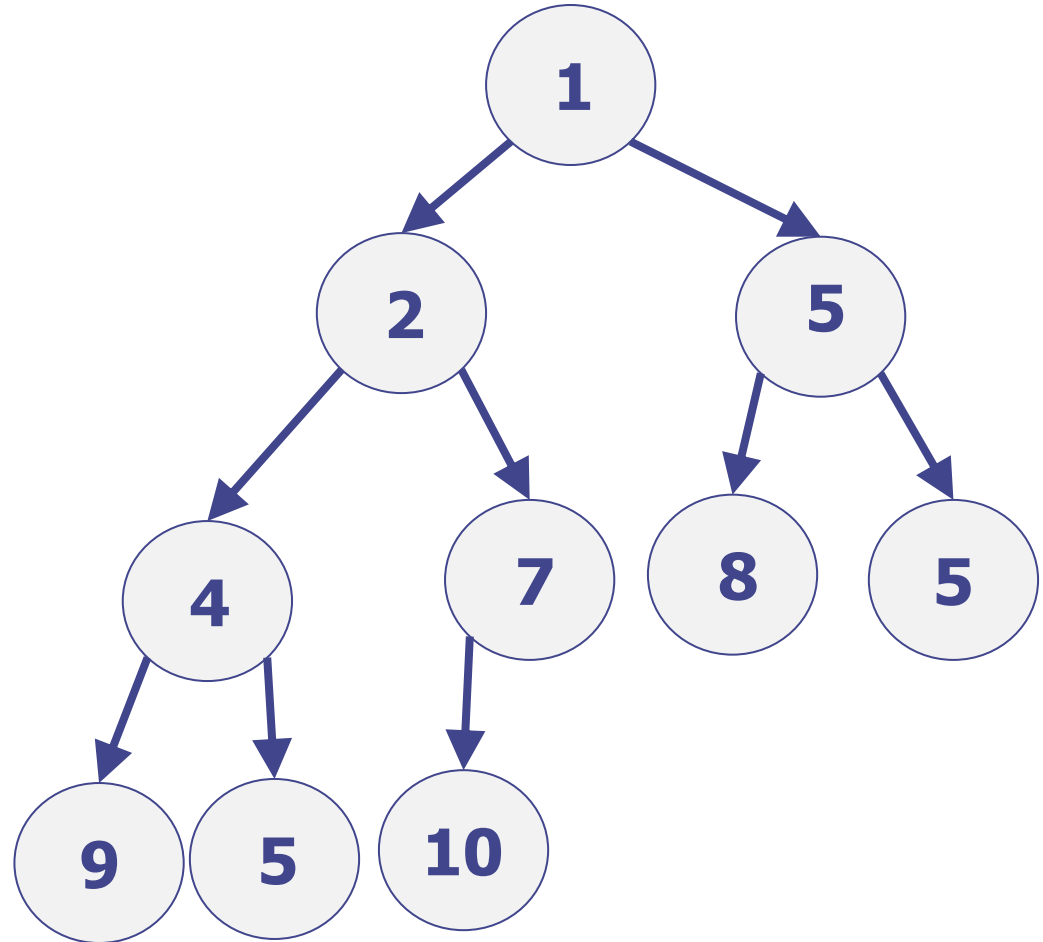
Is this a Heap ?

- No
- Not complete
- Array version would have a gap



Heap Example : Stored as array

- Recall from the trees lectures we can store a binary tree in an array
 - We will draw trees, but they might well be stored as arrays
- For complete binary trees, the elements are stored with “no gaps in the array”
 - Very space efficient
 - May well reduce cache misses
- The first entry is a minimum, but in general **the array is NOT sorted**



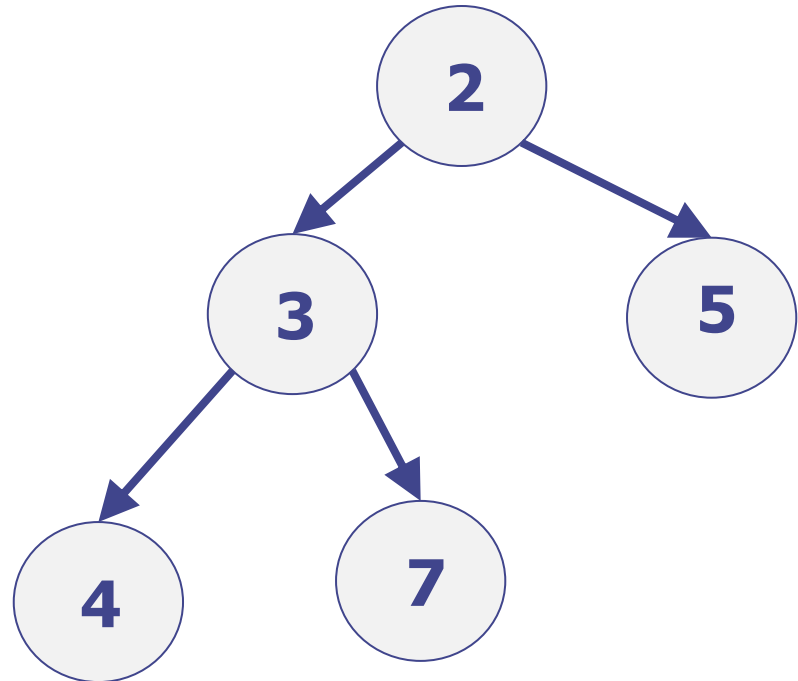
-	1	2	5	4	7	8	5	9	5	10

Heap: Add operation

- Suppose we want to add a new key
- The number of nodes increases by one
 - We want to retain that the tree is complete
 - This implies we add a new node after the last node – or start a new row if needed
- In general, we then need to repair the numeric properties

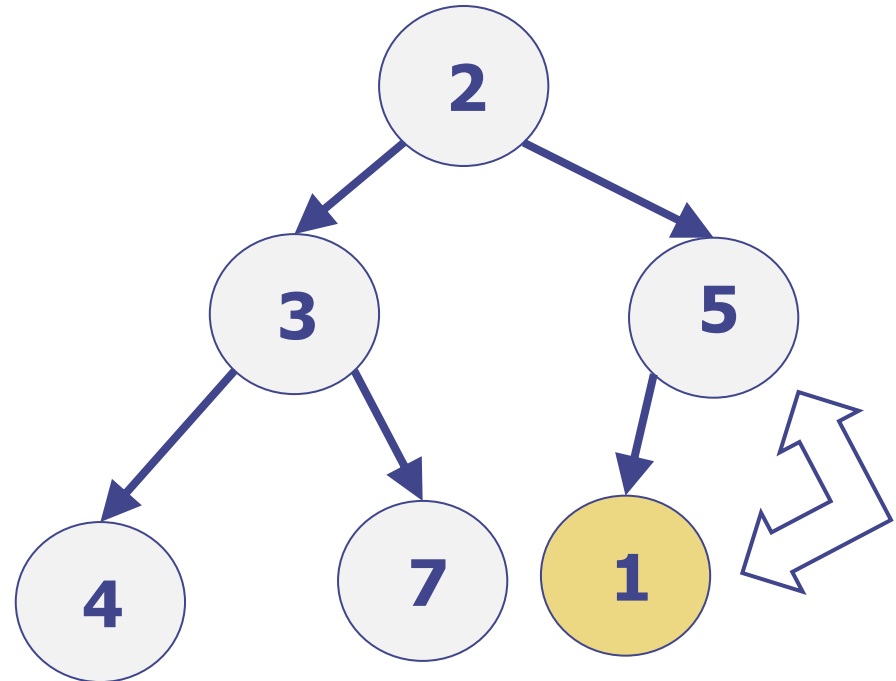
Heap: Add example:

- Suppose we must add(1)
- The last node is "7"
- Must place it as "after 7" i.e. as left child of "5"



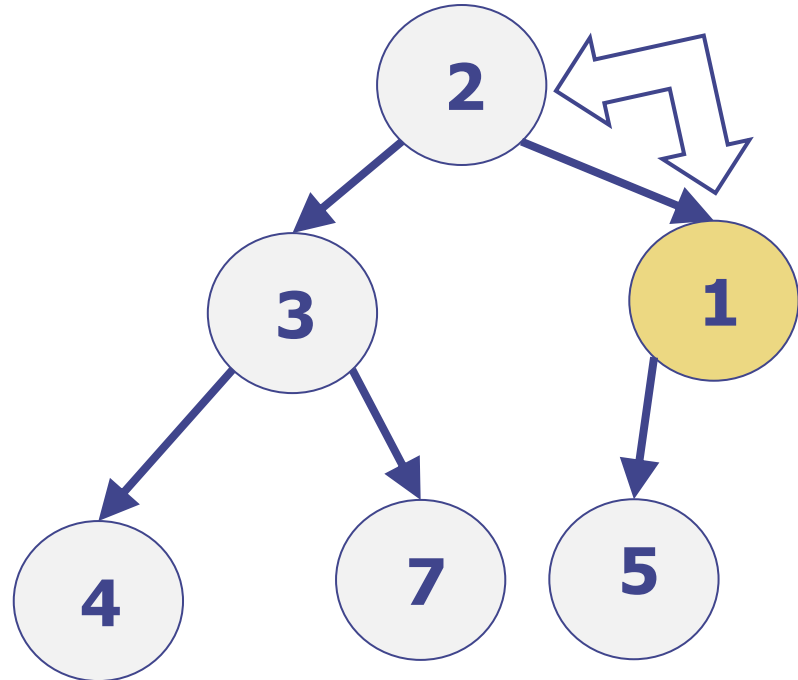
Heap: Add example: "Upheap"

- Add(1) initially breaks the heap property
 - 1 is a child of 5 and should not be smaller
- Immediate repair:
Swap them:



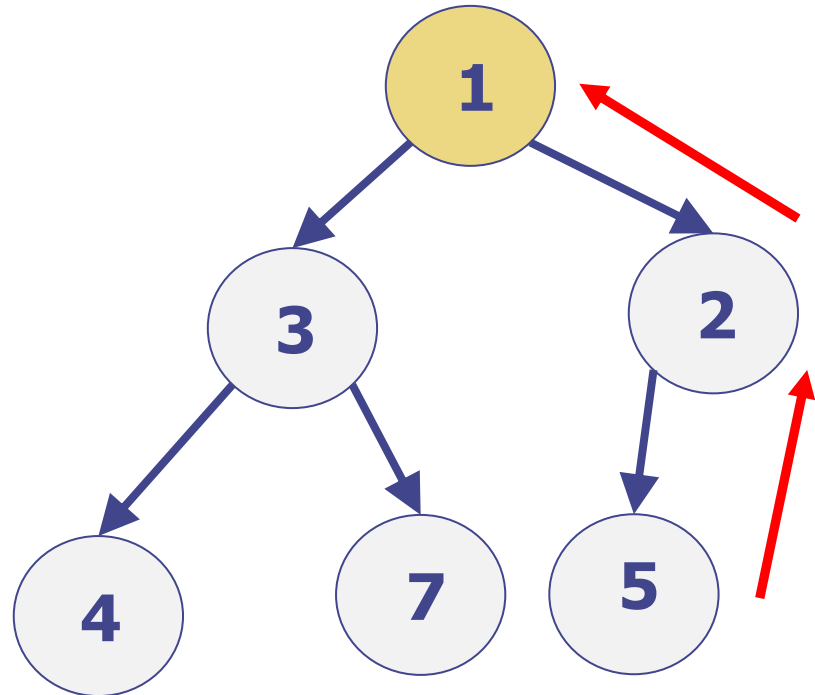
Heap: Add example: "Upheap"

- Heap property is still broken as 2 has a smaller child
- So do another swap



Heap: “Add” example: “Upheap”

- The last swap moved fixed the numeric heap property
 - This time it moved all the way to the root
 - The new node starts as a leaf at the bottom and may move to the top
- Total number of possible swaps is height h of the tree
 - The add takes $O(h)$
- But for complete trees we know height is $O(\log n)$
- Hence: **add is only $O(\log n)$**



Exercises

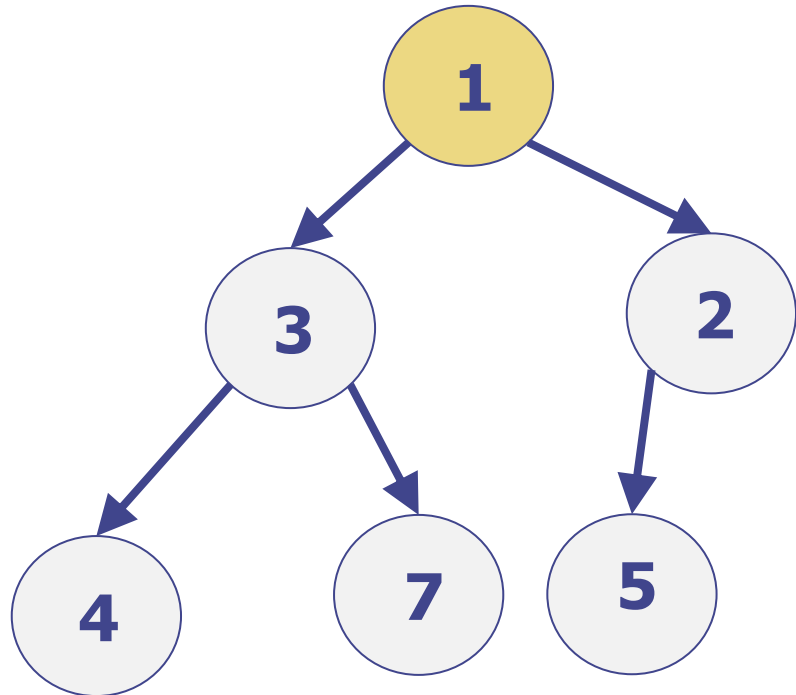
- Convince yourself that the upheap method is correct
 - i.e. that it always results in the heap properties being satisfied after it is completed
 - It is typical in data structure algorithms that required properties are temporarily broken, but then restored
- Remark:
 - You should always strive to convince yourself of the correctness for all algorithms given to you
 - If nothing else, it encourages deeper thinking about the algorithm, and this makes it easier to
 - 'fill in the gaps' if forget parts of them 😊
 - Extend the ideas to other cases

Heap: removeMin

- We need to “remove the root”
 - But we want to keep the shape property – and so need to remove a node
- This suggest to leave the root in place, but remove the last node
 - We will return the original key-value of the root, but overwrite it with the contents of the removed last node

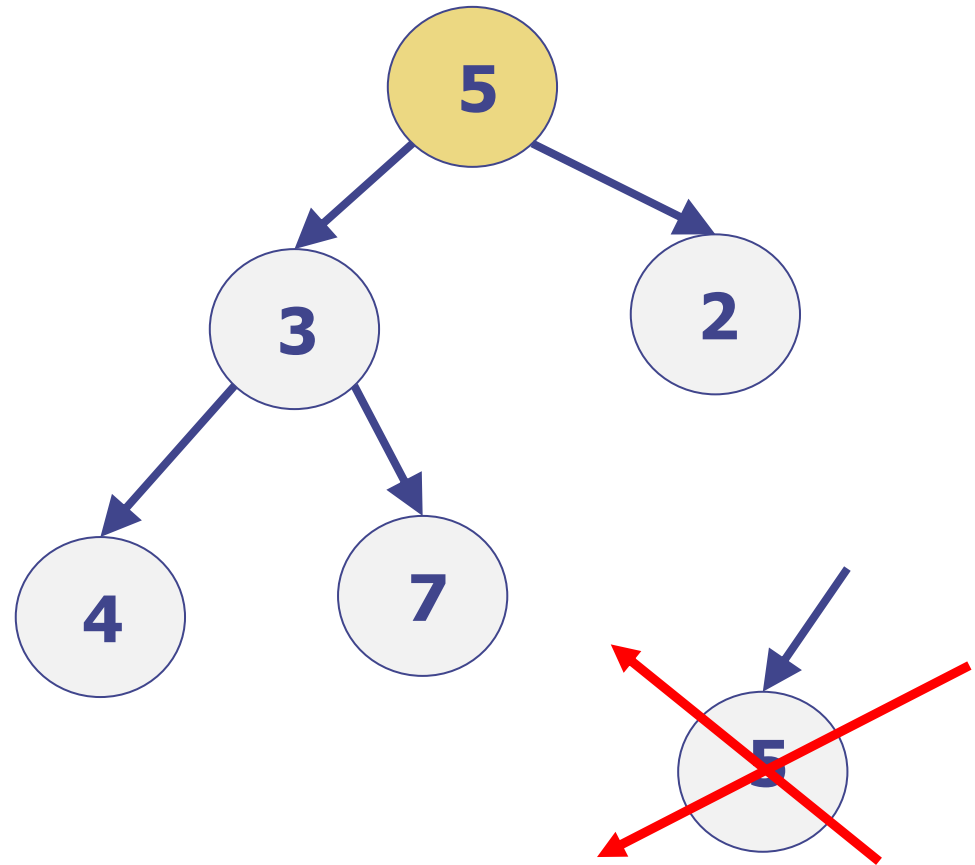
“RemoveMin” example

- Make a copy of the key-value of the root and return them at the end
- Copy the contents of the last node into the root node
- Remove the last node
- Gives:



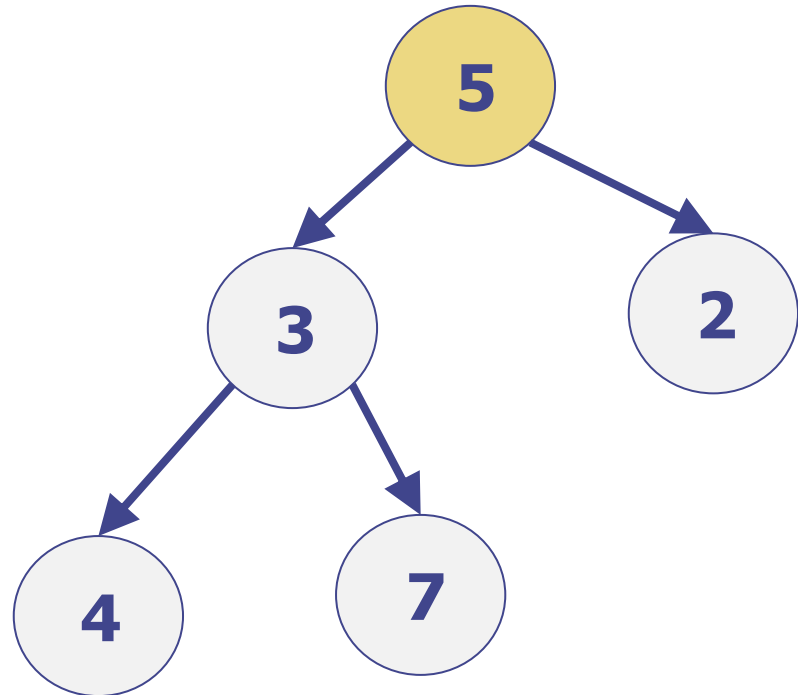
“RemoveMin” example: “DownHeap”

- We make a copy of the key-value root, and return it when finished
- Then we overwrite the root with the values in the last node – in this case 5
- Then removed the last node
 - In the array version this just means decrementing the number of stored elements
 - Do not need to actually remove a node



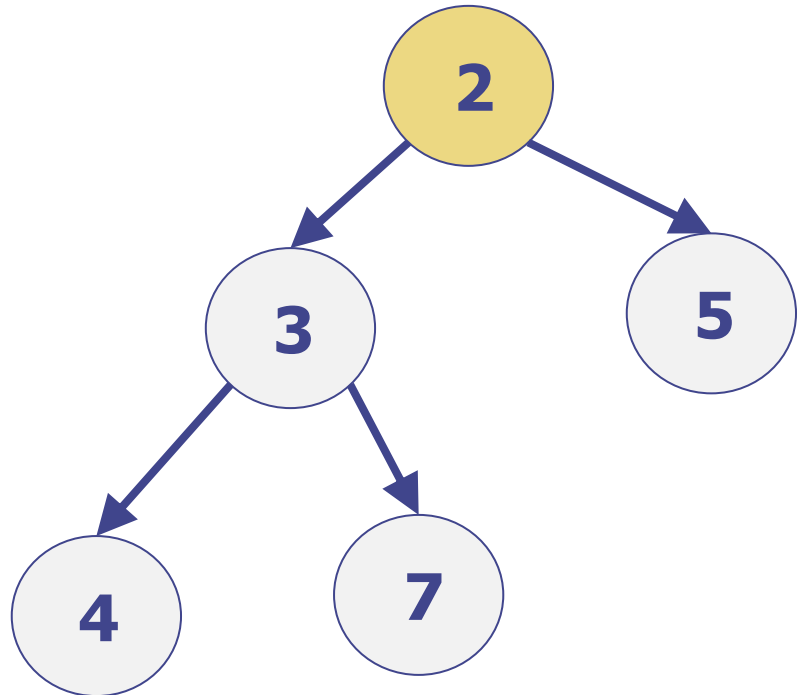
“RemoveMin” example: “DownHeap”

- We have again broken the numeric property
- We will need to move 5 downwards
 - Do we swap 5 with 3?
 - or swap 5 with 2?
 - or does it not matter?
- Observe: If we swap 5 and 3 then the heap property is not restored
- In the trio of 5,3,2 the 2 must become the parent
 - So we swap 5 and 2
 - **The downheap swaps with the smallest child**



“RemoveMin” example: “DownHeap”

- The downheap restores the heap properties
- But the value – which started as the last node – will just follow a path downwards
- Hence the path is no more than the height h
- Hence removeMin is $O(h)$
- Again, it is also $O(\log n)$



Exercise (offline)

- Be sure that you understand why the upheap and downheap operations are correct.
- In particular, why do we only need to fix nodes on the path from the “adjusted nodes” back up to the root node?
 - I.e. Why do we not need to scan the entire tree?
 - Otherwise, the operations would not be $O(h)$ or $O(\log n)$, but would be $O(n)$
 - (that is, they would be exponentially slower)

Heaps operations are $O(\log n)$

- To do “add(k)”
 - Place new node after the last node
 - Do Upheap
 - Cost is $O(\log n)$
- To do “removeMin()”
 - Remove last node and overwrite root
 - Do Downheap
 - Cost is $O(\log n)$
- This is much better than the list-based implementation

Heapsort

In $O(n \log n)$, we can sort an array of n numbers starting from an empty heap:

1. Sequentially insert all the numbers into the heap

- Cost: $O(n \log n)$
- As have n insertions of $O(\log n)$ each

2. RemoveMin until empty

- Cost: $O(n \log n)$
 - As have n removals of $O(\log n)$ each
 - Copy each element back into the array as we go
- Not in-place, as uses the heap as a workspace

Other types of heaps exist

- The previous definition of “heap” is a standard one for a “Binary Heap”
 - It is used/assumed in this module.
- Other notions of “heap” do exist:
 - they keep the numeric ordering of parent relative to children
 - but can have different conditions for the “shape”
 - E.g. “Binomial” and “Fibonacci” heaps
 - Some can even be a forest of binary trees
 - All maintain that: height is $O(\log n)$ as this is crucial for the efficiency

Conclusion

- Priority Queue ADT can be implemented using an unsorted list, a sorted list, or a heap.
 - In the first two cases, one of the methods requires $O(n)$ time.
 - For the heap implementation, all methods run in $O(\log n)$.

IMPORTANT

Do NOT confuse

**“Binary Tree implementations of heaps”
(this lecture – used for PQ ADT)**

with

**“Binary Search Trees”
(next lectures – used for Map ADT)**

Despite saying this every year, some people still get it wrong in assessments – this will lose many marks

Minimal Expectations

- Thoroughly know the definitions
 - Be able to reason about them and explain their motivations, not only be able to reproduce them
- Know the Binary tree implementation
 - **Do NOT confuse “Binary Tree implementations of heaps” with “Binary Search Trees” (later lectures)**
- Know
 - The insertion/deletion procedures
 - Array-based implementations